

1 Lesson 25-09-26

Let's continue talking about instruction set architectures. We recall that this part is a preliminary step in the design of every processor. The instruction set itself is, we said, the minimum set of information that the manufacturing company provides to users (programmers) of its processors. All the rest of the design remains confidential to the processor company (logical and physical architecture, ...).

Control flow instructions determine the control flow in the program. They can be distinguished in four categories:

- **Conditional branches:** this involves a comparison of values, which if evaluated to a true value, triggers a jump to some other location in code. Otherwise nothing happens;
- **jumps:** these are jumps to arbitrary code which always happen;
- **Procedure calls:** these involve saving some state about the CPU before jumping;
- **Procedure returns:** these involve recovering the state saved by procedure calls.

Conditional branches are by far the most frequently executed control flow instructions.

Control flow instructions affect the value of the *program counter* register, which points to the current instruction. The destination address is normally specified as a displacement with respect to the current value of the program counter. In this way:

- We save bits, since the target instruction is often close to the source one;
- The code is position-independent (**PIC**, *Position Independent Code*).

We should make a note about **register** based indirect jumps and procedure calls. They allow:

- To write code including jumps whose target is not known at compile time;
- To implement case or switch statements
- To support dynamically shared libraries (i.e., libraries which are loaded only when called)
- to support virtual functions (i.e., calling different functions depending on the data type).

We not show a table which compares jumps in multiple architectures.

Name	Examples	Test	Advantages	Disadvantages
Condition code (CC)	80x86, ARM, PowerPC, SPARC, SuperH	Tests special bits set by ALU operations, possibly under program control	Sometimes condition is set for free.	CC is extra state. Condition codes constrain the ordering of instructions because they pass information from one instruction to a branch
Condition register/limited comparison	Alpha, MIPS	Tests arbitrary register with the result of a simple comparison (equality or zero tests)	Simple	Limited compare may affect critical path or require extra comparison for general condition
Compare and branch	PA-RISC, VAX, RISC-V	Compare is part of the branch. Fairly general compares are allowed (greater than, less than)	One instruction rather than two for a branch	May set critical path for branch instructions

Let's now focus on **procedures**. When calling a procedure, as we said, some information need to be saved:

- The **return address**: this is the address of the instruction *right after* the call instruction. *Where* this address is stored depends on the architecture:
 - x86 performs *stack* saving, accessing memory at the address pointed to by the stack pointer and decrementing it. This is referred to as *pushing* the address: *popping* the address involves incrementing the stack pointer and recovering the address;
 - Architectures like RISC-V implement **JAL** (*Jump And Link*), where the return address is stored in a register. This saves one memory access for leaf procedure calls (that is procedures which don't call other procedures). If more calls are needed stack or other register saving can be performed manually.
- The accessed registers, based on whether our **ABI** (*Application Binary Interface*) defines *caller saving* or *callee saving* for certain registers (both can be defined, on different registers).
 - **Caller saved** registers are meant to be saved before the procedure call, if the caller intends to keep their value;
 - **Callee saved** registers are meant to remain unchanged after a procedure call. The called procedure can implement this by saving registers itself (through other registers or stack pushing), or by not using them at all.

To summarize instruction sets, we can say that:

- Few instructions are responsible for most of the execution time: load, store, add, subtract, move, register-register, and, shift, compare equal, compare not equal, branch, jump, call, and return.

- PC-relative branch displacements of at least 8 bits are the best choice.
- Register-indirect and PC-relative addressing can also be used in procedure call and return

1.0.1 Instruction set encodings

Instructions have to be encoded in a format the processor can *fetch* and *decoded* before executing. Instruction set **encoding** depends on:

- Which instructions compose the *instruction set*;
- Which *addressing modes* are supported (we have talked about register and memory addressing modes in section 2.2.2).

When a high number of addressing modes is supported, an address specifier field is used to specify the addressing mode and the registers which are possibly involved (this is the CISC philosophy). When the number of addressing modes is low, they can be encoded together with the opcode (this is the RISC philosophy).

We have 3 main types of instruction encoding:

- **Variable** encoding, where instructions have no fixed size and can grow based on immediates, displacements, ... This is the approach used by x86 and VAX architectures.

Other characteristics are:

- Supports any number of operands;
- Instructions have a variable length;
- Lower performance;
- Minimum code size.

- **Fixed** encoding, where instructions are fixed in size and have fields which usually mean the same thing across instructions. This is the approach used by many RISC architectures, such as RISC-V, ARM, MIPS, PowerPC and SPARC.

Other characteristics are:

- Fixed number of operands;
- Address specifier included in the opcode;
- Fixed instruction length;
- Maximum performance;
- Larger code size.

- **Hybrid** encoding, where instructions are mostly fixed, but shorter instructions (often called *compressed* instructions) might appear for memory saving. This solves the problem of many RISC architectures, where the additional instructions that have to be included to perform operations grow code size (we talked about this in 1.2.4). This is the approach used by many compressed extensions for RISC architectures, such as RISC-V compressed (RV32IC), ARM Thumb2, microMIPS, and even older architectures like IBM 360/370.

Other characteristics are:

- Multiple formats (specified by the opcode);
- Allow trading-off between code size and performance.

We show a summarizing figure:

Operation and no. of operands	Address specifier 1	Address field 1	...	Address specifier n	Address field n
----------------------------------	------------------------	--------------------	-----	--------------------------	----------------------

(A) Variable (e.g., Intel 80x86, VAX)

Operation	Address field 1	Address field 2	Address field 3
-----------	--------------------	--------------------	--------------------

(B) Fixed (e.g., RISC-V, ARM, MIPS, PowerPC, SPARC)

Operation	Address specifier	Address field
-----------	-------------------	---------------

Operation	Address specifier 1	Address specifier 2	Address field
-----------	---------------------	---------------------	---------------

Operation	Address specifier	Address field 1	Address field 2
-----------	-------------------	-----------------	-----------------

(C) Hybrid (e.g., RISC-V Compressed (RV32IC), IBM 360/370, microMIPS, Arm Thumb2)

We note that, when designing encodings, the designer should address several conflicting issues:

- The code size;
- The size of the Instruction Set, the number of addressing modes, and the number of registers;
- The complexity of the fetch and decoding hardware.

We should also note that nowadays, assembly-level programs are now produced by compilers, only. This eases the pressure on programmers, but means that the CPU designer and the compiler writer must interact and cooperate.

For instance, choosing which variables have to be put in which registers and when, is one of the crucial optimization phases in a compiler. This problem is based on graph coloring and can be solved easily if the number of registers is high (> 16). Of course, maximum performance would be achieved with an infinite number of registers, but this cannot be achieved as silicon is limited.

Optimizing variable access time by allocating variables to registers is only possible for those stored in the stack or for global variables in memory. It is impossible for variables belonging to the heap, due to the aliasing problem (i.e., the access to a variable is done through pointers).

To summarize, the CPU architect can help the compiler developer by:

- Making the frequent case fast and the rare case correct;
- Regularity;

- Providing primitives, not solutions;
- Simplifying trade-offs among alternatives;
- Providing instructions that bind the quantities known at compile time as constants.

The baseline recommendations are those of:

- **Orthogonality**, that is designing instructions that do not care about the *registers* which are used, or other specific constraints. Many famous instruction sets such as x86 are *not* orthogonal: for instance, multiplication and division instructions only operate on a specific subset of registers, in specific roles;
- **Simplicity**,
- Having at least 16 registers;

1.1 RISC-V introduction

We now give an introduction to the **RISC-V**. This and *ARM* will be the two architectures we cover during the course.

RISC-V is an open standard **ISA** (*Instruction Set Architecture*), having a few instructions in 3 different versions: 32, 64 and 128 bits. The fact that the ISA is fully open means that it is:

- Extremely easy and cheap to implement;
- Customizable and extensible.

The RISC-V ISA itself is defined as a base ISA (RV32I) plus extensions and user customizations. The base extensions feature things such as multiplication, protection, floating point operations, ...

Today, RISC-V is:

- Widely thought in universities;
- Increasingly adopted by companies (e.g., in the automotive domain);
- Strongly supported by the European Commission.

1.1.1 Technical characteristics

Let's get technical. Some generalities about RISC-V are:

- Freely licensed open standard;
- **Load/store** architecture;
- Fixed **32-bit** instruction set (whether RV32I, RV64I or RV128I is implemented);
- Additional **ISA extensions**, for example for security, floating point, and vector operations;
- The supported **addressing modes** are:
 - Displacement;

- Immediate;
- Register indirect.

The supported **data types** are:

- Byte (8 bits);
- Half Words (16 bits);
- Words (32 bits);
- Double Words (64 bits);
- 32-bit single precision floating-point;
- 64-bit double precision floating-point.

In regards to **encoding**, RISC-V allows a **variable** encoding in multiples of 16-bit blocks if code size reduction or specific implementations are needed.

Fixed-length 32-bit instructions are used in the base versions:

- 32-bit instructions have their lowest two bits set to 11;
- 16-bit instructions may have their lowest two bits equal to 00, 01, or 10.

In the case of 32-bit instructions, code memory alignment is a constraint; in other cases, the alignment is relaxed.

A little-endian memory system is the default choice.

1.1.2 Extensions

It counts with three base instructions sets plus a variety of optional extensions. For example, a 64-bit RISC-V possible ISA is defined as RV64IMAFD, known as RV64G, general-purpose scalar instruction set. The 32-bit version is called RV32G.

A table of common extensions is shown below.

Name of base or extension	Functionality
RV32I	Base 32-bit integer instruction set with 32 registers
RV32E	Base 32-bit instruction set but with only 16 registers; intended for very low-end embedded applications
RV64I	Base 64-bit instruction set; all registers are 64-bits, and instructions to move 64-bit from/to the registers (LD and SD) are added
M	Adds integer multiply and divide instructions
A	Adds atomic instructions needed for concurrent processing; see Chapter 5
F	Adds single precision (32-bit) IEEE floating point, includes 32 32-bit floating point registers, instructions to load and store those registers and operate on them
D	Extends floating point to double precision, 64-bit, making the registers 64-bits, adding instructions to load, store, and operate on the registers
Q	Further extends floating point to add support for quad precision, adding 128-bit operations
L	Adds support for 64- and 128-bit decimal floating point for the IEEE standard
C	Defines a compressed version of the instruction set intended for small-memory-sized embedded applications. Defines 16-bit versions of common RV32I instructions
V	A future extension to support vector operations (see Chapter 4)
B	A future extension to support operations on bit fields
T	A future extension to support transactional memory
P	An extension to support packed SIMD instructions: see Chapter 4
RV128I	A future base instruction set providing a 128-bit address space

1.1.3 Registers

The RV64G register bank consists of a register bank (or register file) of 32 elements, that is 32 integer registers $x_0, x_1, \dots, x_{31}$. The same amount of registers is available for floating point single (32-bit) and/or double precision (64-bit).

Register x_0 (called *register zero*) always returns the value 0, and cannot be written to. One reason for having such a register is shown below (3.1.4).

Other registers have roles defined by the ABI (though when doing bare metal we choose to respect them or not). We show a figure below of these roles.

Register	ABI Name	Description	Register	ABI Name	Description
x0	zero	hardwired zero	x16	a6	function argument 6
x1	ra	return address	x17	a7	function argument 7
x2	sp	stack pointer	x18	s2	saved register 2
x3	gp	global pointer	x19	s3	saved register 3
x4	tp	thread pointer	x20	s4	saved register 4
x5	t0	temporary register 0	x21	s5	saved register 5
x6	t1	temporary register 1	x22	s6	saved register 6
x7	t2	temporary register 2	x23	s7	saved register 7
x8	s0 / fp	saved register 0 / frame pointer	x24	s8	saved register 8
x9	s1	saved register 1	x25	s9	saved register 9
x10	a0	function argument 0 / return value 0	x26	s10	saved register 10
x11	a1	function argument 1 / return value 1	x27	s11	saved register 11
x12	a2	function argument 2	x28	t3	temporary register 3
x13	a3	function argument 3	x29	t4	temporary register 4
x14	a4	function argument 4	x30	t5	temporary register 5
x15	a5	function argument 5	x31	t6	temporary register 6

1.1.4 Addressing modes

Addressing modes for data transfers are *immediate* and *displacement*, both counting with an encoded field of 12 bits.

- **Immediate** addressing is done through the usual syntax, remembering that we have 3 registers per instruction (destination, source 1 and source 2, as per RISC ABIs): `addi x1, x2, 32`;
- **Displacement** is specified on indirect register addressing in load/store instructions: `lw x1, 32(x2)`. Let us also note that the `w` in `lw` stands for "word", as in load/store instructions in x86 fashion need their operand size specified in the opcode mnemonic. Of course a displacement of 0 equals to register indirect addressing. An interesting thing is how absolute displacements are specified: we have to include a register for all load instructions, and we can use just the 12 bit displacement as an address if we make that address `x0` (which we know always evaluates to 0).

1.1.5 Instruction formats

Instruction formats are an integral part of the RISC-V ABI. The *green card* available at https://www.cs.sfu.ca/~ashriram/Courses/CS295/assets/notebooks/RISCV/RISCV_CARD.pdf remains an extremely useful resource.

31	27	26	25	24	20	19	15	14	12	11	7	6	0	
funct7				rs2		rs1		funct3		rd		opcode		R-type
imm[11:0]						rs1		funct3		rd		opcode		I-type
imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode		S-type
imm[12 10:5]				rs2		rs1		funct3		imm[4:1 11]		opcode		B-type
imm[31:12]										rd		opcode		U-type
imm[20 10:1 11 19:12]										rd		opcode		J-type

Each instruction belongs to one of 5 formats, which specifies the semantics of fields within the instruction encoding. We see that fields like the instruction *opcode*, and the *funct3* and *funct7* values are shared between most formats.

- **R-type:** these are *register-to-register* instructions.

Field	Description
opcode	7 bit primary operation opcode
rd	5 bit specifier for destination register
funct3	Specifies the type of operation (<code>add</code> / <code>and</code> / ...)
rs1	5 bit specifier for first register
rs2	5 bit specifier for second register
funct7	Selects a specific suboperation from a main operation (for instance sign extending and non-sign extending shifts)

- **I-type:** these are instructions with *immediates*.

Field	Description
opcode	7 bit primary operation opcode
rd	5 bit specifier for destination register
funct3	Specifies the type of operation (<code>addi</code> / <code>andi</code> / ...)
rs1	5 bit specifier for first register
immediate	12 bit signed immediate used for logical operands, arithmetic signed operands, load/store address byte offsets, and PC-relative branch signed instruction displacement. Note that this is <i>sign extended</i> , as all other immediates in RISC-V! (for simplicity).

- **S-type:** these are *store* instructions.

Field	Description
opcode	7 bit primary operation opcode
rs1	5 bit specifier for register to add to the 12 bits immediate
rs2	5 bit specifier for register to copy in memory
funct3	Specifies the size of the operation
imm fields	These two values compose the 12-bit signed value.

- **B-type:** these are *branch* (basically conditional additions affecting the program counter) instructions.

Field	Description
opcode	7 bit primary operation opcode
rs1	5 bit specifier for first register to compare
rs2	5 bit specifier for second register to compare
funct3	Specifies the type of comparison
imm fields	These two values compose the 12 bit signed value.

- **U-type:** these are so-called *upper immediate* instructions, used to target the upper 20 bits 12 bit immediate instructions can't reach.

Field	Description
opcode	7 bit primary operation opcode
rd	5 bit specifier for destination register (usually the program counter)
imm fields	These two values compose the 20 bit signed value.

- **J-type:** these are *jump* instructions.

Field	Decription
opcode	7 bit primary operation opcode
rd	5 bit specifier for destination register (usually the program counter)
imm fields	These two values compose the 20 bit signed value.

1.1.6 Available instructions

The instructions available on the RV32G, grouped by function:

- Load and store
- ALU operations
- Branches and Jumps
- Floating Point
- Miscellaneous

In the following some of the most used instructions and pseudo-instructions are described.

Load/store: as we know, RISC-V processors use a load-store architecture. The main memory is accessed only through load and store instructions.

Example instruction	Instruction name	Meaning
ld x1,80(x2)	Load doubleword	$\text{Regs}[x1] \leftarrow \text{Mem}[80 + \text{Regs}[x2]]$
lw x1,60(x2)	Load word	$\text{Regs}[x1] \leftarrow_{64} \text{Mem}[60 + \text{Regs}[x2]]_0^{32} \text{ ##}$ $\text{Mem}[60 + \text{Regs}[x2]]$
lwu x1,60(x2)	Load word unsigned	$\text{Regs}[x1] \leftarrow_{64} 0^{32} \text{ ##} \text{Mem}[60 + \text{Regs}[x2]]$
lb x1,40(x3)	Load byte	$\text{Regs}[x1] \leftarrow_{64} (\text{Mem}[40 + \text{Regs}[x3]]_0^{56} \text{ ##}$ $\text{Mem}[40 + \text{Regs}[x3]])$
lbu x1,40(x3)	Load byte unsigned	$\text{Regs}[x1] \leftarrow_{64} 0^{56} \text{ ##} \text{Mem}[40 + \text{Regs}[x3]]$
lh x1,40(x3)	Load half word	$\text{Regs}[x1] \leftarrow_{64} (\text{Mem}[40 + \text{Regs}[x3]]_0^{48} \text{ ##}$ $\text{Mem}[40 + \text{Regs}[x3]])$
flw f0,50(x3)	Load FP single	$\text{Regs}[f0] \leftarrow_{64} \text{Mem}[50 + \text{Regs}[x3]] \text{ ##} 0^{32}$
fld f0,50(x2)	Load FP double	$\text{Regs}[f0] \leftarrow_{64} \text{Mem}[50 + \text{Regs}[x2]]$
sd x2,400(x3)	Store double	$\text{Mem}[400 + \text{Regs}[x3]] \leftarrow_{64} \text{Regs}[x2]$
sw x3,500(x4)	Store word	$\text{Mem}[500 + \text{Regs}[x4]] \leftarrow_{32} \text{Regs}[x3]_{32..63}$
fsw f0,40(x3)	Store FP single	$\text{Mem}[40 + \text{Regs}[x3]] \leftarrow_{32} \text{Regs}[f0]_{0..31}$
fsd f0,40(x3)	Store FP double	$\text{Mem}[40 + \text{Regs}[x3]] \leftarrow_{64} \text{Regs}[f0]$
sh x3,502(x2)	Store half	$\text{Mem}[502 + \text{Regs}[x2]] \leftarrow_{16} \text{Regs}[x3]_{48..63}$
sb x2,41(x3)	Store byte	$\text{Mem}[41 + \text{Regs}[x3]] \leftarrow_8 \text{Regs}[x2]_{56..63}$

Pseudo-instructions are not actual RISC-V instructions but they are often more convenient for the programmer. The assembler converts them to real RISC-V instructions (one or more).

One important pseudo-instruction is `li`, which stands for *load immediate*. As we know this operation is not straightforward in RISC-V because immediates are only 12 bits.

- For small immediates (which fit into 12 bits) the conversion is straightforward:

```

1 li t0, 4
2
3 # translates to:
4
5 addi t0, zero, 4

```

- For big immediates (which don't fit into 12 bits) the conversion needs an upper immediate instruction:

```

1 li t0, 0xFEDC8765
2
3 # translates to:
4
5 lui t0, 0xFEDC8 # upper 20 bits
6 addi t0, t0, 0x765 # lower 12 bits

```

ALU operations are performed on operands held in the processor registers (we also called them register-to-register operations).

Instruction types are:

- Immediate and three-operand instructions;
- Shift instructions;
- Multiply and divide instructions if the M extension is available.

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)	Note
add	ADD	R	0110011	0x0	0x00	rd = rs1 + rs2	
sub	SUB	R	0110011	0x0	0x20	rd = rs1 - rs2	
xor	XOR	R	0110011	0x4	0x00	rd = rs1 ^ rs2	
or	OR	R	0110011	0x6	0x00	rd = rs1 rs2	
and	AND	R	0110011	0x7	0x00	rd = rs1 & rs2	
sll	Shift Left Logical	R	0110011	0x1	0x00	rd = rs1 << rs2	
srl	Shift Right Logical	R	0110011	0x5	0x00	rd = rs1 >> rs2	
sra	Shift Right Arith*	R	0110011	0x5	0x20	rd = rs1 >> rs2	msb-extends
slt	Set Less Than	R	0110011	0x2	0x00	rd = (rs1 < rs2)?1:0	
sltu	Set Less Than (U)	R	0110011	0x3	0x00	rd = (rs1 < rs2)?1:0	zero-extends

Some interesting pseudo-instructions we note are **seqz** and **snez**:

- **seqz** sets a destination register if the source register is equal to 0:

```

1 seqz x1, x2
2
3 # translates to:
4
5 sltiu x1, x2, 1 # if (x2 == 0) x1 <- 1
6                # else x1 <- 0

```

- **snez** sets a destination register if the source register is not equal to 0:

```

1 seqz x1, x2
2
3 # translates to:
4
5 sltu x1, x0, x2 # if (x2 != 0) x1 <- 1
6                # else x1 <- 0

```

Branches and jumps: these comprise control transfer instructions (which are mainly unconditional), including:

- PC-relative jumps;
- Jump to register;
- Conditional branches.

jal	Jump And Link	J	1101111			rd = PC+4; PC += imm	
jalr	Jump And Link Reg	I	1100111	0x0		rd = PC+4; PC = rs1 + imm	

- `j label`: a PC relative jump having a $\pm 1\text{MiB}$ range. It is a pseudo-instruction:

```
1 j label
2
3 # translates to:
4
5 jal x0, label
```

- `jal label`: a PC relative jump and link having a $\pm 1\text{MiB}$ range. It is a pseudo-instruction which assumes x1 to be the return address:

```
1 jal label
2
3 # translates to:
4
5 jal x1, label
```

- `jal x3, label`: a PC relative jump and link having a $\pm 1\text{MiB}$ range;
- `jr x3`: an indirect jump to register with $\pm 2\text{KiB}$ plus register range. It is a pseudo-instruction:

```
1 jr x3
2
3 # translates to:
4
5 jalr x0, 0(x3)
```

- `jalr x3`: an indirect jump to register with $\pm 2\text{KiB}$ plus register range. It is a pseudo-instruction which assumes x1 to be the return address:

```
1 jalr x3
2
3 # translates to:
4
5 jalr x1, 0(x3)
```

- `jalr x3, x2, 0`: an indirect jump and link to register. Saves the return value in the destination register.
- `call sub1`: calls a named subroutine. It is a pseudo-instruction which assumes x1 to be the return address:

```
1 call sub1
2
3 # translates to:
4
5 auipc x1, sub1 # top 20 bits of subroutine address
6 jalr x1, offset (x1)
```

- `ret`: returns from a subroutine. It is a pseudo-instruction which assumes `x1` to be the return address:

```
1 ret
2
3 # translates to:
4
5 jalr x0, 0(x1)
```